CS 2430 – Programming Project 3

Team 2 Project Plan: Space Shuttle Payload Optimization

Team Members: Lia Day, Matthew Meek, Rowan Bailey
Course: CS 2430
Project: Programming Project 3 – Space Shuttle Payload Optimization
Primary Language: Java

1. Project Overview

The goal of this project is to determine which combination of twelve scientific experiments should be selected for a space shuttle payload with a maximum weight capacity of 700 kg. Each experiment has an assigned weight and rating. The objective is to maximize the overall rating without exceeding the available payload capacity.

The team will implement three greedy algorithms and a brute-force algorithm to compare different approaches to solving the optimization problem. The results will demonstrate the differences between making locally optimal decisions and finding the globally optimal solution.

2. Proposed Program Structure

The program will be organized into four primary components:

Experiment Class

The Experiment class will represent an individual scientific experiment. Each experiment will contain:

- Experiment name
- Weight in kilograms
- Rating

The twelve experiments will be stored in a collection of Experiment objects that can be accessed by the different algorithms.

Greedy Algorithms

The greedy algorithms will be implemented as three separate methods:

- highestRatingFirst()
- lightestFirst()
- bestRatioFirst()

Each method will evaluate the experiments according to its respective selection strategy and add experiments to the payload as long as the total weight does not exceed 700 kg.

Brute-Force Algorithm

The brute-force algorithm will generate and evaluate all 4,096 possible subsets of experiments. It will calculate the total weight and rating of each subset, discard invalid combinations, and identify the three highest-rated valid solutions.

Main Class

The Main class will initialize the experiments, execute each algorithm, and display the results. It will also provide a comparison of the greedy solutions against the optimal brute-force solution.

3. Team Responsibilities

Each team member will serve as the lead for a specific area of the project. Responsibilities will be divided to ensure that implementation, verification, and communication are handled throughout development.

Lia Day - Implementation Lead

- Lead the overall implementation of the project.
- Establish the program structure and coordinate the development of the Experiment class, greedy algorithms, brute-force algorithm, and Main class.
- Ensure that the different components work together correctly.
- Maintain code organization, readability, and documentation.
- Coordinate implementation progress and address integration issues.

Matthew Meek - Verification Lead

- Lead testing and verification of the project.
- Develop test cases for the greedy algorithms and brute-force implementation.
- Verify that experiment selections never exceed the 700 kg weight limit.
- Check that the brute-force algorithm evaluates all 4,096 possible subsets.
- Compare greedy algorithm results against the optimal brute-force solution.
- Identify potential errors, edge cases, and inconsistencies in the program.

Rowan Bailey - Communication Lead

- Coordinate communication between team members throughout the project.
- Organize team discussions and communicate project updates.
- Keep track of assigned responsibilities, progress, and upcoming milestones.
- Coordinate documentation and ensure that project requirements are addressed.
- Assist with preparing the final team report and screencast.
- Ensure that each team member's contributions are documented and communicated.

All team members will participate in code reviews, collaborative problem-solving, testing, and final project preparation. Although each member has a designated lead role, the team will work together to complete the project and ensure that all requirements are met.

4. Verification and Testing Approach

The team will verify that each algorithm follows its intended selection strategy and never exceeds the maximum payload capacity.

For the highestRatingFirst() method, experiments are sorted by rating in descending order and added to the payload as long as the total weight remains less than or equal to 700 kg.

For example, the two experiments with ratings of 9 have weights of 264 kg and 170 kg, giving a combined weight of 434 kg. After selecting these experiments, 266 kg of capacity remains. There are three experiments with ratings of 8 and weights of 203 kg, 104 kg, and 65 kg. If the 203 kg experiment is considered first, it will be selected, bringing the total weight to 637 kg. Neither of the remaining rating-8 experiments will fit. However, if the 104 kg and 65 kg experiments are considered first, both can be selected, resulting in a combined weight of 603 kg and a higher total rating.

This demonstrates that when multiple experiments have the same rating, the order in which they are considered can affect the final result. The team will establish consistent tie-breaking rules and test how they affect the greedy algorithm's output.

The other greedy algorithms will be verified using the same general approach. The lightest-first strategy will prioritize experiments with the lowest weight, while the ratio-first strategy will calculate each experiment's rating-to-weight ratio and prioritize the highest values.

The brute-force algorithm will serve as the reference for verifying the optimal solution. Since it evaluates every possible subset, the team can compare the greedy solutions against the highest-rated valid combinations.

Additional testing will include:

- Confirming that no solution exceeds 700 kg.
- Testing an empty payload and individual experiment selections.
- Testing combinations that total exactly 700 kg.
- Confirming that all 4,096 possible subsets are evaluated.
- Confirming that the three highest-rated valid subsets are distinct.
- Comparing the results of each greedy strategy against the brute-force solution.

5. Project Milestones

Milestone	Tasks
1. Project Setup	Establish project structure, create the Experiment class, and initialize the experiment data.
2. Greedy Implementation	Implement and test all three greedy strategies.
3. Brute-Force Implementation	Generate all subsets and identify the top three valid solutions.
4. Integration	Connect the algorithms through the Main class and standardize output formatting.
5. Verification	Test edge cases, compare algorithm results, and verify the optimal solution.
6. Documentation	Complete the report, README, and contribution documentation.
7. Final Submission	Record the screencast, export the repository ZIP, and submit all required files.

6. Expected Deliverables

The team will produce the following deliverables:

Program

- Experiment data structure.
- Three working greedy algorithms.
- Working brute-force algorithm.
- Output displaying the top three valid subsets.
- Comparison summary identifying which greedy strategies match the optimal solution.
- Dynamic programming pseudocode comments or an optional dynamic programming implementation.

Documentation

- Project README with program instructions.
- Documentation describing the algorithms and their approaches.
- Testing and verification documentation.
- CONTRIBUTIONS.md identifying individual responsibilities.
- Formal team report of 3–5 pages.

Final Submission

- Team report in PDF or DOCX format.
- Repository ZIP containing source code, documentation, README, and contribution records.
- Screencast demonstrating the project and each team member's contribution.

7. Project Goals

The primary goal is to successfully implement and compare the three greedy algorithms and brute-force exhaustive search. The team will use the results to evaluate how different selection strategies affect the overall payload rating and why locally optimal decisions do not always produce the globally optimal solution.

The team also aims to produce readable, modular, and well-documented code that can be understood and maintained by other students. If time permits, the team will explore a dynamic programming implementation for extra credit and compare its efficiency with brute-force search.

Through this project, the team will gain a better understanding of greedy algorithms, exhaustive search, optimization problems, and the tradeoffs between algorithm efficiency and solution accuracy.